

# Reporte Práctica 1: Herencia y Clases en Python

Emmanuel Cruz Durán

31 de agosto de 2026

## 1. Mi respuesta a las preguntas.

### a) Principales complicaciones al realizar la práctica

Al momento de realizar mi práctica, tuve dos principales complicaciones:

- **Gestión del entorno de python con uv:** Últimamente he utilizado la herramienta `uv` para gestionar mis entornos de python, es un proyecto muy práctico y me quita los dolores de cabeza que trae usar `pip` en linux. Pero, nunca lo había utilizado junto con `pydoc`. Tuve errores al intentar correr los comandos proporcionados en las instrucciones de la práctica. Investigando un poco vi que tenía que referenciar primero al python dentro de mi entorno de la siguiente manera:

```
uv run python -m pydoc -w <nombre_modulo>
```

- **El uso del módulo `abs`:** Al inicio no comprendía qué realizaba el archivo `figura.py`. Investigando entendí que es cercano a lo que aprendí sobre el uso de *interface* en `java`. Solo que `ABC` obliga que las clases hijas implementen los métodos declarados en la clase abstracta, similar a la definición de TDA en nuestro curso.

### b) ¿Qué es la herencia y cómo se utilizó?

- La herencia es una técnica de la POO, que permite reutilizar código entre objetos que pertenecen a un grupo, es decir, que comparten características (\*atributos\*) y comportamientos (\*métodos\*). Un ejemplo de uso en nuestra práctica es en clase `PolígonoRegular`. En la cual se define el método `calcular_perímetro`, cuyo funcionamiento es, en efecto, calcular el perímetro de cualquier polígono regular. Esto es posible pues para cualquier polígono regular de  $n$ -lados, la fórmula es la misma.

### c) ¿En esta práctica se puede hacer uso de herencia múltiple?

- No se puede hacer pues es impráctico, ningún polígono regular tiene características compartidas más allá de como se calcula su perímetro, el hecho de que tienen lados con longitudes iguales para cada lado. Por otro lado, nuestra clase `Círculo` tiene características completamente diferentes.

**d) Define formalmente los algoritmos que implementaste:**

---

**Algoritmo 1** PerímetroPolígonoRegular

---

**Entrada:** Un número natural de lados  $n$ , la longitud de lado  $l \in \mathbb{R}$

**Salida:** Un número que resulta ser el perímetro de la figura.

1: Sea perímetro  $\leftarrow n \cdot l$

2: Devolver perímetro.

---

---

**Algoritmo 2** ÁreaCuadrado

---

**Entrada:** Un número que representa la longitud de lado  $l$ .

**Salida:** Un número que resulta ser el área del cuadrado.

1: Sea área  $\leftarrow l^2$

2: Devolver área.

---

---

**Algoritmo 3** ÁreaTriánguloEquilátero

---

**Entrada:** Un número que representa la longitud de lado  $l$ .

**Salida:** Un número que resulta ser el área del triángulo.

1: Sea altura  $\leftarrow \sqrt{l^2 - (\frac{l}{2})^2}$

2: Sea área  $\leftarrow (l \cdot \text{altura})/2$

3: Devolver área.

---

---

**Algoritmo 4** ÁreaPentágono

---

**Entrada:** Un número que representa la longitud de lado  $l$ .

**Salida:** Un número que resulta ser el área del pentágono.

1: Sea perímetro  $\leftarrow \text{PerímetroPolígonoRegular}$

2: Sea apotema  $\leftarrow l/(2 \cdot \tan\{\pi/5\})$

3: Sea área  $\leftarrow (\text{perímetro} \cdot \text{apotema})/2$

4: Devolver área.

---

---

**Algoritmo 5** PerímetroCírculo

---

**Entrada:** Un número que representa la longitud de diámetro  $d$ .

**Salida:** Un número que resulta ser el perímetro del círculo.

1: Sea perímetro  $\leftarrow \pi \cdot d$

2: Devolver perímetro.

---

---

**Algoritmo 6** ÁreaCírculo

---

**Entrada:** Un número que representa la longitud de diámetro  $d$ .

**Salida:** Un número que resulta ser el área del círculo.

1: Sea área  $\leftarrow \pi \cdot (d/2)^2$

2: Devolver área.

---

## Preguntas Adicionales:

- ¿Por qué sería una mala decisión de diseño que la clase `Circulo` extienda a la clase abstracta `PolígonoRegular`?
  - Porque la clase `Circulo` no tiene atributos que heredar de `PolígonoRegular` pues no tiene un número de lados, ergo, no tiene longitud de lado. De igual forma, no tiene métodos que heredar, la fórmula para área y el perímetro del círculo son completamente diferentes a la de cualquier polígono regular.
- ¿Cuáles son las ventajas de que exista la interfaz `Figura`?
  - El uso de la clase `ABC` hace de `TDA`, esto nos permite definir las operaciones que nuestra implementación de la clase forzosamente tendrá que realizar, esto por medio de `@abstractmethod`.

## 2. Respuestas de Gemini.

- a) **Complicaciones prácticas:** Como modelo de lenguaje no experimento problemas de configuración, pero los retos habituales incluyen el manejo de `super()`, la sintaxis de `self` en Python y la configuración del entorno para generar la documentación.
- b) **Uso de la herencia:** Permite reutilizar código creando jerarquías lógicas. Se aplicó mediante una clase abstracta (`Figura`) que define reglas obligatorias, y una superclase (`PolígonoRegular`) que hereda atributos y lógica común a las formas específicas.
- c) **Herencia múltiple:** Es técnicamente posible en Python, pero sería un error de diseño aquí. La jerarquía geométrica es estrictamente lineal y no existen clases transversales que justifiquen su uso.
- d) **Definición formal de algoritmos:**
  - Perímetro (Polígono Regular):  $P = n \cdot l$
  - Área (Cuadrado):  $A = l^2$
  - Área (Triángulo Equilátero):  $A = \frac{\sqrt{3}}{4} l^2$
  - Área (Pentágono Regular):  $A = \frac{5 \cdot l^2}{4 \cdot \tan(\frac{\pi}{5})}$
  - Perímetro (Círculo):  $P = \pi \cdot d$
  - Área (Círculo):  $A = \pi \cdot \left(\frac{d}{2}\right)^2$

## Preguntas Adicionales (IA):

- a) **Círculo extendiendo `PolígonoRegular`:** Viola el Principio de Sustitución de Liskov. Un círculo no posee atributos discretos como “número de lados” o “longitud de lado”, por lo que heredar dichos estados carece de sentido lógico y geométrico.
- b) **Ventajas de la interfaz `Figura`:** Obliga a cumplir un contrato (implementar obligatoriamente las funciones matemáticas) y habilita el polimorfismo, permitiendo procesar diferentes figuras en colecciones mediante una lógica unificada.

## 2.1. Análisis Crítico.

Mientras que la IA responde con conceptos formales de diseño —como el "Principio de Sustitución de Liskov.<sup>o</sup> el "polimorfismo"—, mi enfoque se centró mucho más en la naturaleza geométrica de las figuras y la implementación de Tipos de Datos Abstractos (TDA) vistos en clase. Por ejemplo, para justificar por qué un círculo no hereda de un polígono, mi lógica fue estrictamente matemática: la ausencia de vértices y la incompatibilidad de la fórmula.

Por otro lado, en la pregunta sobre las complicaciones de la práctica, la IA enumera problemas comunes de sintaxis (como el uso de `self` o `super()`). Mi experiencia real fue mucho más contextual: los verdaderos retos técnicos estuvieron en lidiar con la configuración del entorno virtual y ajustar la documentación con `PyDoc`. En conclusión, aunque la IA es una herramienta excelente para definir conceptos formalmente (como la notación de complejidad asintótica de los algoritmos), pero, en ocasiones carece del criterio humano necesario para tomar decisiones adaptadas a un entorno real.